

Cette séquence s'étend sur 4 séances de TP (TP1, TP2, TP3, TP4). Elle a pour objectif de réaliser le développement d'une application « client lourd » avec WPF.

Séquence 1

Codage application cliente

Alexandre GUIDET

Compétence 1	Compétence 3	Compétence 6
☒ Élaborer et implémenter les spécifications ☒ Appliquer des principes d'accessibilité ☒ Adopter de bonnes pratiques	☐ Concevoir et développer des applications communicantes	☐ Appliquer une démarche pour intégrer une équipe informatique
<i>En respectant les besoins décrits par le client</i> <i>En appliquant les principes algorithmiques</i> <i>En veillant à la qualité du code et à sa documentation</i> <i>En choisissant les ressources techniques appropriées</i>	<i>En sécurisant le système d'information</i> <i>En appliquant les normes en vigueur et les bonnes pratiques architecturales et de sécurité</i> <i>En offrant une qualité de service optimale</i> <i>En assurant la continuité d'activité</i>	<i>En inscrivant sa démarche au sein d'une équipe pluridisciplinaire</i> <i>En accompagnant la mise en œuvre des évolutions informatiques</i> <i>En veillant au respect des contraintes juridiques</i> <i>En développant une communication efficace et collaborative</i>

1 INTRODUCTION

L'IUT souhaite se doter d'un outil permettant de gérer les absences des étudiants. L'objectif de cette séquence de TP est de coder une application « client lourd », en WPF, pour PC/Windows.

Une analyse rapide a été effectuée pour cette application et se trouve sur le commun.

La séquence occupe **4 séances de TP** (donc 8h) et sera évaluée **à la fin** de la 4^{ème} séance. Le travail est **individuel**.

La persistance des données utilisera dans un premier temps des simples fichiers JSON¹. Des données exemples sont fournies sur le commun et peuvent être utilisées pour tester l'application. La sauvegarde doit être automatique (chaque action déclenche la sauvegarde).

① *Pensez bien que l'application est destinée à utiliser des données issues d'un service web et qu'il faudra, un jour, la modifier en ce sens...*

Créez un dépôt privé GitHub dans dept-info-iut-dijon appelé 2025-OGA-<votre nom>. Ajoutez votre enseignant de TP à ce dépôt et envoyez-lui l'URL de ce dernier par courriel. Placez dans un sous-dossier **Client** de votre dépôt une application WPF.

Le barème utilisé pour l'évaluation sera double :

- Les **fonctionnalités** apporteront des **points positifs** (si fonctionnelles...). Attention, si rien de marche, il n'y aura aucun point positif...
- Les erreurs de **qualité** apporteront des **points négatifs**
 - Violation de principe SOLID
 - Code moche ou sale (return multiples, noms de variables inappropriés, etc.)
 - Non-respect du cahier des charges
 - Absence de documentation
 - Absence de tests

① *Conseil : faites une **nouvelle branche** quand vous démarrez un cas d'utilisation. Au fur et à mesure de votre avancée, validez quand une étape est franchie (et validée). Une fois celui-ci fonctionnel (il faut bien s'en assurer !) vous pouvez fusionner la branche avec main. Ne fusionnez pas si ça ne fonctionne pas. L'enseignant ne corrigera que la branche principale, les autres seront ignorées.*

¹ Les entités à utiliser sont indiquées dans le document d'analyse, elles serviront ultérieurement à échanger avec le service web

Les maquettes proposées sont indicatives, pour indiquer les fonctionnalités attendues, mais vous pouvez totalement les modifier si vous le souhaitez, du moment que les cas d'utilisation soient bien implémentés.

Vous avez le choix de l'ordre d'implémentation des fonctionnalités, à vous de voir !

① *Aucune conception n'est fournie ou imposée, ce qui ne veut pas dire qu'il ne faut pas réfléchir à l'architecture du logiciel... il n'est pas demandé de fournir des diagrammes UML (ce qui ne veut pas dire qu'ils sont inutiles).*

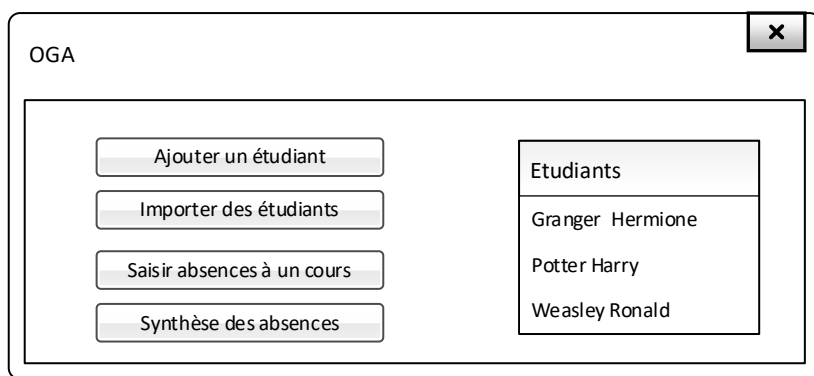
2 MAQUETTES INDICATIVES PAR CAS D'USAGE

Vous trouverez sur le commun des fichiers contenant des extraits de code XAML pouvant être utilisés pour gagner du temps. Attention : les contrôles ne sont pas nommés, aucun *binding* de donnée n'est fait, aucun évènement n'est prévu, le code est bien évidemment à adapter !

2.1 LISTER LES ÉTUDIANTS

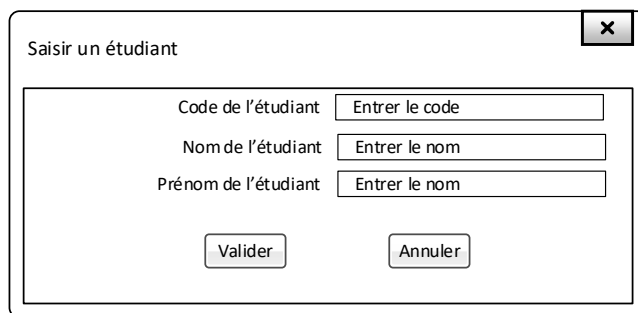
La fenêtre principale peut contenir la liste des étudiants, cette liste sera mise à jour à chaque fois que c'est nécessaire (démarrage, ajout d'un étudiant...).

La fenêtre peut aussi comprendre des actions pour les cas d'utilisation de 1^{er} niveau.



Maquette de la fenêtre principale OGA. La fenêtre a un titre "OGA" et un bouton de fermeture "x". À l'intérieur, il y a quatre boutons à gauche : "Ajouter un étudiant", "Importer des étudiants", "Saisir absences à un cours", et "Synthèse des absences". À droite, il y a une liste intitulée "Etudiants" contenant les noms : "Granger Hermione", "Potter Harry", et "Weasley Ronald".

2.2 SAISIR UN ÉTUDIANT



Maquette de la fenêtre "Saisir un étudiant". La fenêtre a un titre "Saisir un étudiant" et un bouton de fermeture "x". À l'intérieur, il y a trois champs de saisie : "Code de l'étudiant" avec le placeholder "Entrer le code", "Nom de l'étudiant" avec le placeholder "Entrer le nom", et "Prénom de l'étudiant" avec le placeholder "Entrer le nom". En bas, il y a deux boutons : "Valider" et "Annuler".

Le code de l'étudiant ne peut pas être vide, de même que son nom. La validation entraîne la fermeture de la fenêtre et la mise à jour de la liste dans la vue principale (ainsi que la sauvegarde automatique). L'annulation entraîne la fermeture de la fenêtre, sans modification.

2.3 IMPORTER UNE LISTE D'ÉTUDIANTS

Le clic sur le bouton déclenche l'ouverture d'une boîte d'ouverture de fichier standard (OpenFileDialog), puis l'importation elle-même, ce qui déclenche un message dans une fenêtre surgissante (MessageBox) de type « importation terminée : x étudiants importés » et la mise à jour de la liste des étudiants, ainsi que la sauvegarde automatique. Seul le format CSV est demandé pour l'importation.

Le fichier CSV source aura le format suivant :

- Valeurs délimitées par des points-virgules
- Pas de ligne d'entête
- Champs Code, Nom, Prénom (dans cet ordre)
- D'autres champs peuvent être fournis après, ils sont à ignorer

Un exemple de CSV est fourni sur le commun.

2.4 SAISIR UNE ABSENCE

Saisie d'une absence à un cours

Date du cours

Heure du cours

Durée :

Enseignant :

Matière :

Etudiants absents :
☐ Harry Potter
☐ Hermione Granger
☒ Drago Malefoy
☐ Ron Weasley

La liste doit présenter l'ensemble des étudiants et permettre de saisir pour chacun le fait qu'il soit absent ou non. La durée du cours doit être exprimée en minutes (par défaut 120). Le nom de l'enseignant et celui de la matière sont des textes libres.

2.5 LIRE LES ABSENCES D'UN ÉTUDIANT

A partir de la liste des étudiants, si l'utilisateur clique sur un étudiant cela doit montrer (dans une nouvelle vue par exemple) la liste de ses absences :

Absences d'un étudiant

Etudiant :

Nom et prénom de l'étudiant

Absences

17/02/2025, 10h, Maths (M. Megot), 120 min

20/02/2025, 8:00, Algo (Mlle Chiffre), 60 min

20/02/2025, 10:00, Anglais (M. Machin), 90 min

2.6 RÉSUMER LES ABSENCES

Une nouvelle vue présentera une liste par étudiant synthétisant le nombre d'absences.

Résumé des absences

Harry Potter, 24h dont 12h non justifiées

Hermione Granger, 2h dont 0h non justifiées

Ron Weasley, 10h dont 10h non justifiées